SUPPORTING DOCS

Brand AI-Readiness Audit & Query Intelligence

Complete technical supporting evidence, design rationale, and case-coverage record

Project: Brand AI-Readiness Audit + NLP Query Intelligence

Document: Supporting Docs (final)

Scope: Repository-verified design journey, NLP/evidence architecture, case inventory, worked examples, and research foundation

Prepared for: Hackathon evaluation / judging

Table of Contents

0. How to read this document

1. What This Project Is

2. System Architecture

3. The Design Journey

4. The NLP / Semantic Reasoning Layer

5. Evidence Matching

6. Query Generation and Difficulty

7. Domain Generalization

8. Complete Case Inventory

9. Master Case Table

10. Worked Examples

11. What We Tried

12. Real-World Website / Domain Motivation

13. Failure Mode Table

14. Implemented vs. Tested vs. Considered — Summary View

15. Final Case Summary

16. Limitations

17. Research Foundation

18. Master Traceability Table

Appendix A — Test File Index (Query Intelligence)

Appendix B — Fixture Index

Brand AI-Readiness Audit + Query Intelligence Technical supporting evidence, design rationale, and case-coverage record

> **Status: complete.** Stage 1 (repository-based case and NLP analysis) and Stage 2 (research integration, Section 17) are both finished. Everything in this document is drawn directly from the repository — source code, tests, fixtures, and project documentation — or from the three research sources supplied by the project owner, with provenance stated for every claim.

---

0. How to read this document

Every non-obvious claim in this document carries a **origin tag** so a reader can tell what kind of evidence backs it:

Tag	Meaning
`[CODE]`	Read directly from source, cited as `file:line`
`[TEST]`	A specific, named automated test proves this
`[FIXTURE]`	A checked-in synthetic test fixture demonstrates this
`[REAL RUN]`	Output of an actual audit run against a live website
`[DOC]`	Quoted from the project's own README/SKILL.md/reference docs
`[ILLUSTRATIVE]`	An example built for this document, not a recorded observation

Status labels used throughout (see §19 for the full legend): **IMPLEMENTED+TESTED**, **IMPLEMENTED+PARTIALLY TESTED**, **CONSIDERED / DESIGN CASE**, **ILLUSTRATIVE ONLY**, **NOT CURRENTLY IMPLEMENTED**.

---

1. What This Project Is

There are two subsystems, one read-only dependency direction:

```
Brand AI-Readiness Audit   (9 Agent Skills, network-touching, standalone)
          │
          │   snapshot.json / report.json  —  READ ONLY
          ▼
Query Intelligence         (separate subsystem, reads the audit's output)
```

Brand AI-Readiness Audit answers: *can an AI assistant find, read, render, understand, and cite this brand's website at all* — crawl access, renderability, structured data, quotability, freshness, on-site follow-through `[DOC: brand-ai-readiness-audit/README.md]`.

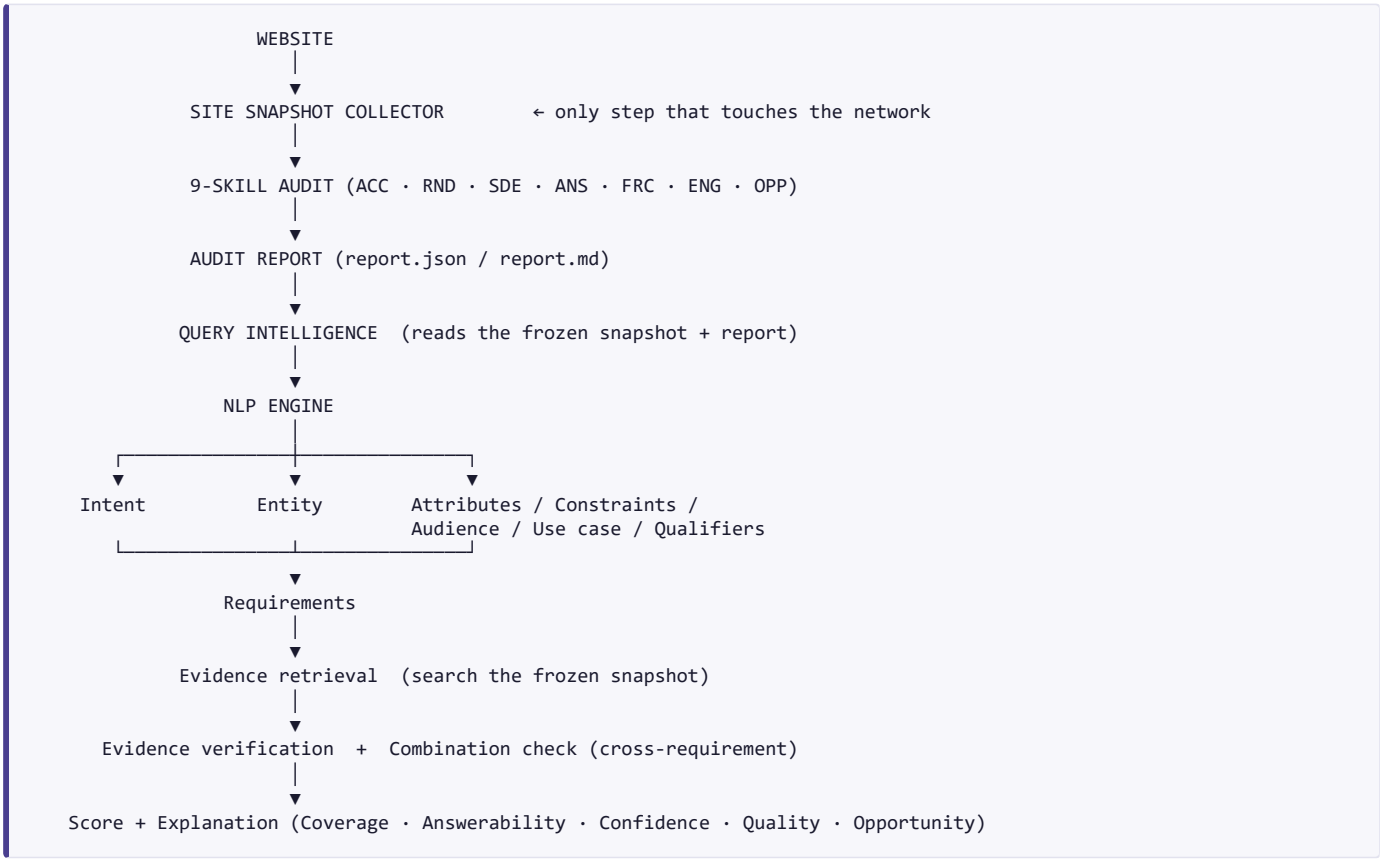
Query Intelligence answers a narrower, harder question: *given one specific natural-language query a real user might type, can this particular website's own already-captured content actually support that answer?* `[DOC: nlp-query-intelligence/README.md]` This second question is the one this document spends most of its time on, because it is where the project's NLP/semantic reasoning lives.

The project's own stated problem (quoted verbatim):

> "A website can genuinely contain the information a user wants and still fail an AI assistant: the fact might be locked in an image, blocked by robots.txt for retrieval bots, contradicted elsewhere on the site, or simply never tied together into one place." `[DOC: README.md]`

---

2. System Architecture



`[DOC: README.md — "System architecture" diagram, reproduced with the module-level detail confirmed by reading lib/schema.py]`

Query Intelligence is **not** a second crawler. It "makes no network requests of its own — evidence comes only from fields already captured in `snapshot.json`" `[DOC: nlp-query-intelligence/README.md]`, and the snapshot contract itself is declared **frozen**: "Every detector is a pure function over this file... Never fetch. If a detector needs something the snapshot lacks, that dimension is `not_assessed`. It is not a finding." `[DOC: skills/site-snapshot-collector/references/snapshot-contract.md]`

---

3. The Design Journey

There is no git history available in this workspace to mine for commit-by- commit evolution, so this section reconstructs iteration honestly from what the code itself documents: several modules contain explicit "used to / previously / this is the fix for" language describing a prior approach, the problem it caused, and the change made. These are marked `[CODE — evolution comment]` below; nothing here is invented.

#	Initial approach	Problem discovered	Design change	Proof it stuck
1	Comparison queries ("Laptop vs Keyboard") only ever detected the *first* entity	"a second product name in a comparison query had no representation and its words leaked one-by-one into `unrecognized_term`" `[CODE: nlp_engine.py:91-111]`	Added `_detect_comparison_entity`, gated so it **only** runs when `intent == "comparison"` — it never fires on an ordinary single-entity query	`test_comparison_query_recognizes_both_entities`, `test_comparison_entity_only_searched_for_when_intent_is_comparison` `[TEST]`

#	Initial approach	Problem discovered	Design change	Proof it stuck
2	Bare attribute mentions with no value ("laptop with wheels")	"This used to be silently dropped entirely" `[CODE: nlp_engine.py:179-203]` — violating the project's own hard rule that no query term may vanish silently	`_extract_bare_attributes` added as a first-class, presence-only requirement category	`test_no_term_is_ever_silently_dropped` `[TEST]`
3	Entity candidacy drawn from page titles **and** H2/H3 feature-bullet text	Feature words like "shortcuts"/"backlighting" were becoming false entity candidates	Restricted entity candidacy to page titles and structured-data `Product.name` only `[CODE: entity_schema.py:1-12]`	`test_generic_marketing_modifier_is_not_treated_as_an_entity` `[TEST]`
4	A single site-wide "excluded words" list built from all domain vocabularies combined	Real regression: "business" (a shopping audience word) deleted "Business Administration" as a candidate entity on an education site; "science" broke "Computer Science"	`concept_words_for(domain_type)` scopes exclusions to **one domain's own vocabulary only**, never a cross-domain union `[CODE: domain_vocabulary.py:150-171]`	`test_concept_words_are_domain_scoped_not_a_global_union` `[TEST]`
5	`INVALID_REQUIREMENT` treated like `NOT_ASSESSABLE` (excluded from the scoring denominator)	A query like "Keyboard with RAM" could read as 100% coverage because the invalid pairing was excused rather than counted as a gap	Scoring explicitly keeps `INVALID_REQUIREMENT` in the denominator with a `0.0` multiplier — described in-repo as "the exact bug the rework fixes" `[CODE: test_scoring.py]`	`test_invalid_requirement_is_NOT_excluded_like_not_assessable` `[TEST]`
6	Each requirement (price, RAM, audience) verified independently against the whole set of entity pages	Independently-true facts from **different products** could combine into a false 100%-coverage answer — a "Frankenstein" result. Documented as "the exact bug the query-generation quality review found" `[CODE: test_combination_checker.py]`	`combination_checker.py` added as a post-hoc joint-satisfiability pass over already-produced evidence	`test_combination_checker.py` (see §6.3) `[TEST]`
7	Generated price-constraint queries used an "80% of max" threshold	Comment: "a threshold that only the priciest listings would ever fail is technically a constraint but tests nothing" `[CODE: query_templates.py:26-31]`	Switched to the **median** observed price, which "genuinely splits the catalog"	`test_query_generator.py` quality-gate tests `[TEST]`
8	Query-generation templates drew attributes/audiences from a **site-wide** union	This is what would make "keyboard with RAM" generatable on a site that also sells laptops	Templates are generated **per entity**, from that entity's own `entity_schemas[entity]` only `[CODE: query_templates.py:1-9]`	`test_mixed_site_never_generates_a_cross_entity_attribute_query` `[TEST]`

#	Initial approach	Problem discovered	Design change	Proof it stuck
9	Audit report schema: `effort` enum missing `"content"`, `owner` enum missing `"marketing"`	"found and fixed during end-to-end integration testing" `[DOC: brand-ai-readiness-audit/README.md]`	Narrow, additive, backward-compatible schema corrections	Stated in project README's own "Status" log `[DOC]`
10	One opportunity category, "template-leverage", covering both hand-written cross-detector relationships and generic same-template observations	"it made it impossible to tell, from category alone, whether an opportunity came from a hand-written relationship or a generic same-template observation" `[DOC: brand-ai-readiness-audit/README.md]`	Split into `template-leverage` and `cross-detector-template`	Stated in project README `[DOC]`

The throughline across all ten iterations: **a case was found where the naive version silently lost information or silently manufactured false confidence, and the fix always made the failure loud (a new status, a new requirement, a scoped lookup) rather than papering over it.** That principle — "never drop a term, never invent evidence, always show the gap" — is the closest thing this project has to a founding design decision, and it recurs in almost every section below.

---

4. The NLP / Semantic Reasoning Layer

4.1 Why NLP Was Needed

Take the query used throughout this project's own tests and user manual:

> "Best laptop for CSE students under ₹60,000 with 16GB RAM and good battery life" `[TEST: test_nlp_engine.test_full_query_extraction]` `[DOC: docs/USER_MANUAL.md]`

A keyword search for the words `laptop`, `students`, `60000`, `16GB`, `RAM`, `battery` against a page of text cannot distinguish:

```
"laptop"            → an ENTITY the query is about
"16GB RAM"          → an ATTRIBUTE + a VALUE, tied to that entity
"under ₹60,000"     → a NUMERIC CONSTRAINT on a different field (price)
"CSE students"      → an AUDIENCE, not a product spec
"good battery life" → an ATTRIBUTE with a QUALITATIVE, not numeric, requirement
"best"              → a QUALIFIER expressing recommendation INTENT, not a testable fact
```

Nor can it tell whether "16GB RAM" and "under ₹60,000" are true of the *same* product or of two different ones on the page (§6.3). This is the entire reason a requirement/evidence pipeline exists instead of a substring search: **the system needs to know what role each fragment of the query plays before it can decide what evidence would even count as an answer.**

To be precise about what "understanding" means here: this is **deterministic, explainable, hybrid semantic query analysis** — normalization plus regex/pattern rules plus a controlled vocabulary taxonomy, with **no external model and no API call**: "every extraction here is deterministic and traceable to an explicit rule" `[CODE: nlp_engine.py:1-13]`. It is not a claim of human-level language understanding, and the project is explicit that it never fabricates a claim beyond what its own rules can justify (§4.8, §12).

4.2 The NLP Pipeline

```
Natural-Language Query
          │
          ▼
      Normalization            (normalizer.py — unicode fold, lowercase, tokenize)
          │
          ▼
       Intent                  (nlp_engine._detect_intent — recommendation / comparison /
          │                     availability / specification / general)
          ▼
        Entity                 (nlp_engine._detect_entity — matched against the site's
          │                     OWN discovered entities, longest-phrase-first)
          ▼
  Attributes / Values          (valued: "16GB RAM"; bare: "with wheels")
          │
          ▼
 Constraints / Audience /
 Use Case / Qualifiers         (price cues, audience/use-case synonym tables, qualifier list)
          │
          ▼
     Requirements              (one record per concept: {id, category, key, raw})
          │
          ▼
 Requirement Validation        (requirement_validator.py — is this attribute even a
          │                     real property of THIS entity, per the site's own evidence?)
          ▼
  Evidence Retrieval           (evidence_index.py — pages that mention the entity)
          │
          ▼
 Evidence Verification         (evidence_matcher.py — per-category matcher, word-boundary
          │                     phrase matching, ±char-window context checks)
          ▼
 Combination Check             (combination_checker.py — are ALL requirements true of the
          │                     SAME product, not scattered across several?)
          ▼
 Coverage / Answerability /
 Confidence / Quality /
 Opportunity                   (scoring.py, query_quality.py, opportunity.py)
          │
          ▼
 Explanation                    (recommendations.py, finding_correlator.py — "why", not
                                 just "what")
```

`[CODE: lib/schema.py:108-161 build_query_result — orchestration order]`

4.3 Why the System Represents a Query This Way

NLP concept	Example	Why needed	Failure without it	How it's handled	Test/fixture
Intent	"best" → recommendation; " vs " → comparison	Different intents change what counts as a satisfying answer (a comparison needs two entities, not one)	"Laptop for students" could be wrongly read as a comparison because it contains "for"	`_detect_intent` checks a fixed-priority marker list: comparison → availability → specification → recommendation → general `[CODE: nlp_engine.py:69-78]`	`test_comparison_intent_not_triggered_by_the_word_for`, `test_comparison_intent_triggered_by_vs`
Entity	"laptop"	Every other concept must be checked *against* something; without an entity there's nothing to scope evidence to	"best desktop tower" on a laptop-only site would otherwise silently match laptop evidence	Matched only against `profile["entities"]` — the site's *own* discovered entities, longest-text-first	`test_entity_not_found_for_a_product_never_mentioned`
Attribute + Value	"16GB RAM"	A number alone is meaningless; it must be tied to a named property	"16" could mean RAM, storage, or nothing	Unit → attribute mapping (`gb→ram/storage`, disambiguated by a synonym word in a ±15-char window)	`test_ram_attribute_resolved_by_context`
Bare attribute (no value)	"laptop with wheels"	A concept can be mentioned without a number and still must not vanish	Silent term-dropping (§3, iteration 2)	First-class `attribute` requirement with no numeric value	`test_no_term_is_ever_silently_dropped`
Numeric constraint	"under ₹60,000"	Distinguishing a ceiling from a floor from an approximate figure changes what evidence satisfies it	"under" and "over" price cues would otherwise be indistinguishable from a bare price mention	Classified via a 20-char pre-match cue window into `price_max` / `price_min` / `price_approx` / `price_mentioned`	`test_price_max_cue`, `test_price_min_cue`
Audience	"CSE students"	A product being *about* students (audience fit) is different from a product merely being *near* the word "students"	The "student discount" page problem (§5.2)	Synonym-table match + false-context guard	`test_student_discount_page_alone_does_not_satisfy_audience_relevance`
Use case	"for programming"	Same category of problem as audience, for activity-fit rather than person-fit	A laptop page mentioning "programming" once in passing could wrongly validate every programming-oriented query	Synonym-table match scoped to the entity's own pages	domain_vocabulary use-case tables, `entity_schema.py`
Qualifier	"best", "premium", "cheap"	Some qualifiers are testable against real evidence phrases; others are not, and treating them the same would either under- or over-claim	Treating "best" as verifiable would fabricate a pass/fail for an inherently subjective superlative	`QUALITY_QUALIFIERS`: `premium` / `budget` have phrase-evidence lists; `best/top/great/good` map to `None` (§4.8)	`test_best_is_flagged_unverifiable`
Requirement (the unifying record)	`{id, category, key, raw}`	Every concept above must become one uniform, addressable unit before validation/evidence can run over it generically	Without a common shape, each concept type would need its own bespoke downstream logic	One `requirements` list, one validator, one evidence dispatcher keyed by `category`	`test_full_query_extraction`

NLP concept	Example	Why needed	Failure without it	How it's handled	Test/fixture
Relationship (entity↔attribute)	RAM *belongs to* laptop, not keyboard	A recognized concept ("RAM is a real attribute somewhere on the web") says nothing about whether it is a genuine property of *this* entity	"Keyboard with RAM" silently reading as `NOT_FOUND` (looks like the site is missing content) or, worse, `SUPPORTED` from leaked laptop evidence	`requirement_validator.py` checks the key against `entity_schemas[entity]["attributes"]`, built only from pages mentioning that entity	`test_keyboard_with_ram_is_invalid_requirement_not_a_false_pass`

4.4 Keyword Matching vs. Our Approach

Naive keyword matching:

```
Query contains "RAM"
Website contains "RAM"
        ↓
   SUPPORTED
```

The real, confirmed failure this produces: on the `mixed-category-site` fixture, a wireless-keyboard product page's own body text reads "*Popular with professionals and programmers*" `[FIXTURE: mixed-category-site/snapshot.json]`. The substring `"ram"` is literally present inside `"programmers"`. A naive `"ram" in page_text` check would report RAM evidence on a page that never mentions RAM at all.

The project's own normalizer module states this as a documented correctness requirement, not a style preference:

> "Plain `needle in haystack` substring checks are a real bug, not a simplification — 'ram' is a substring of 'programming', 'pro' of 'professional', 'mat' of 'format'. Every phrase-containment check in this package must go through this [word-boundary function] ... never bare `in`." `[CODE: normalizer.py:59-64]`

Every phrase check in the codebase is required to route through `contains_phrase`, which compiles `\b<escaped phrase>\b` — a whole-word regex, not a substring test. This is proven directly by a regression test built from the real fixture text:

> `test_ram_inside_the_word_programmers_is_not_a_false_match` — on the keyboard page's own text, `contains_phrase(blob, "programmers")` is `True` while `contains_phrase(blob, "ram")` is `False`. `[TEST: test_negative_controls.py]`

Our approach:

```
Query
 ↓
Meaning / components   (intent, entity, attribute, value, constraint, ...)
 ↓
Entity                  (which product/category is this about)
 ↓
Attribute relationship  (is this attribute even valid for that entity?)
 ↓
Requirement             (one addressable, typed unit)
 ↓
Relevant evidence       (only pages that mention the entity, word-boundary matched)
 ↓
Verification            (SUPPORTED / PARTIALLY_SUPPORTED / NOT_FOUND / CONTRADICTED /
                         NOT_ASSESSABLE / INVALID_REQUIREMENT)
```

This is **not** a claim that the system resolves all semantic ambiguity — word-boundary matching plus a ±20/±60-character context window is a deliberately narrow, auditable technique, not full disambiguation (see §18, Limitations, for what it does *not* catch).

4.5 Token → Concept → Relationship → Requirement → Evidence → Verification

Valid chain:

```
"16GB RAM"
      ↓
RAM concept (unit "gb" maps to candidate attributes {ram, storage};
             disambiguated by a nearby synonym word)
      ↓
Laptop → RAM  (requirement_validator: "ram" ∈ entity_schemas["laptop"]["attributes"] → yes)
      ↓
RAM = 16GB requirement   (category=attribute, key=ram, value=16.0)
      ↓
Find laptop evidence     (evidence_index scoped to pages mentioning "laptop")
      ↓
Verify value             (16.0 found and matches → SUPPORTED; found but different → CONTRADICTED)
```

`[TEST: test_attribute_supported_with_matching_value, test_attribute_contradicted_with_different_value]`

Invalid chain, caught rather than silently passed or silently failed:

```
"Keyboard with RAM"
      ↓
RAM concept exists (it's a recognized attribute concept in the shopping vocabulary)
      ↓
Keyboard → RAM  (requirement_validator: "ram" ∈ entity_schemas["keyboard"]["attributes"] → NO,
                  because keyboard's schema was built only from pages that mention "keyboard",
                  and none of them ever discuss RAM)
      ↓
INVALID_REQUIREMENT   (not NOT_FOUND — the distinction matters, see below)
```

`[TEST: test_keyboard_with_ram_is_invalid_requirement_not_a_false_pass — mixed-category-site fixture]`

Why `INVALID_REQUIREMENT` and not `NOT_FOUND`: `NOT_FOUND` means *the site should have this evidence and doesn't* (a real content gap, actionable). `INVALID_REQUIREMENT` means *this property does not apply to this entity at all* (there is nothing to add — the question itself doesn't fit the product). Conflating the two would either wrongly blame the site for a category error in the query, or wrongly excuse a real gap. `opportunity.py` enforces this distinction explicitly: an `INVALID_REQUIREMENT` contributes **zero** actionable-gap points, "because 'Keyboard with RAM' scoring low is not an opportunity — there is nothing to add" `[CODE: opportunity.py:15-20]`, confirmed by `test_invalid_requirement_gap_is_not_an_opportunity`.

4.6 Entity–Attribute Relationships (Major Section)

This is one of the project's strongest technical cases. The problem: recognizing that "RAM" is a known attribute *concept* is not the same as knowing RAM is a property *of this product*.

The `mixed-category-site` fixture `[FIXTURE]` exists specifically to force this distinction:

```
Laptop ("Pro Laptop X"):
  RAM = 16GB          ✓ evidenced
  processor, display  ✓ evidenced

Keyboard ("Wireless Keyboard Pro"):
  backlighting = Yes          ✓ evidenced (RGB backlighting)
  switch_type = mechanical    ✓ evidenced
  RAM                         ✗ never mentioned anywhere on any keyboard page
```

Both product families share marketing vocabulary — both are "Pro", both target "professionals" — which is exactly the trap: a system that disambiguates only by keyword overlap has no way to tell these apart. `entity_schema.py` builds each entity's attribute/audience/use-case/price schema **only from pages that mention that specific entity**, so:

> "'RAM', discovered from laptop pages, can never validate a keyboard requirement, because `entity_schemas["keyboard"]["attributes"]` was built only from pages that mention 'keyboard'." `[CODE: requirement_validator.py:9-12]`

Confirmed directly by fixture-grounded tests:

- `test_ram_belongs_to_laptop_schema_not_keyboard` — RAM is present in the laptop schema, absent from the keyboard schema `[TEST]`
- `test_keyboard_has_its_own_legitimate_attributes` — keyboard's schema legitimately contains `{backlighting, connectivity, switch_type}` `[TEST]`
- `test_students_audience_does_not_leak_from_laptop_to_keyboard` — the same leakage guard applies to audience, not just attributes `[TEST]`
- `test_price_signals_are_scoped_per_entity_not_site_wide` — laptop prices (₹58k–95k) and keyboard prices (₹3.2k–4.5k) never blend into one site-wide range `[TEST]`

So "Keyboard with RAM" does **not** silently inherit the laptop's RAM evidence, and does **not** read as a plain content gap — it is flagged as a category mismatch (§4.5).

4.7 Requirement Construction

For the query "Best laptop for CSE students under ₹60,000 with 16GB RAM", confirmed by `test_full_query_extraction` `[TEST: test_nlp_engine.py]` (which asserts intent, entity, audience, and the exact requirement-category set produced):

```
1. entity            = laptop                    (category: entity)
2. audience           = students                  (category: audience)
3. price_constraint   = price_max, 60000, INR      (category: price_constraint)
4. attribute          = ram, value 16.0           (category: attribute)
5. quality_qualifier  = best (not independently verifiable, see §4.8)
```

Requirement categories actually implemented: `entity`, `price_constraint`, `attribute`, `audience`, `use_case`, `quality_qualifier`, `unrecognized_term`, plus the derived, non-user-stated `combination` category added after evidence-matching (§6.3). Every requirement is given a deterministic, content-addressed id: `"R-%s-%s" % (category[:3].upper(), sha256(...)[:8])` `[CODE: nlp_engine.py:64-66]` — the same query text always produces the same requirement ids, which is what makes the pipeline's determinism testable (`test_generation_is_deterministic`, `test_analyzing_the_same_query_twice_is_deterministic`).

Hard invariant: a word is never dropped for failing to match a pattern — it becomes an `unrecognized_term` requirement instead, resolved to `NOT_ASSESSABLE` rather than silently vanishing `[CODE: nlp_engine.py:1-13]`, confirmed by `test_no_term_is_ever_silently_dropped` ("laptop with wheels" keeps "wheels" as a real, visible requirement).

4.8 Subjective / Qualitative Language

The vocabulary explicitly splits qualifiers into two groups `[CODE: vocabulary.py:29-45]`:

Qualifier	Evidence-phrase list?	Treated as	Rationale (quoted)
`premium`	Yes — `["premium","flagship","high-end","aluminium body","oled display",...]`	Testable requirement	Has concrete, checkable phrase evidence
`budget`	Yes — `["budget","affordable","value for money","entry-level",...]`	Testable requirement	Same
`best`	**No — `None`**	Marked `verifiable: False`	"no single site's own content can verify a superlative claim" `[CODE: vocabulary.py:29-34]`
`top`, `great`, `good`	**No — `None`**	Marked `verifiable: False`	Same reasoning

Confirmed by `test_best_is_flagged_unverifiable` — "best laptop" produces a qualifier with `verifiable: False`, and the matcher resolves it to `NOT_ASSESSABLE` rather than inventing a pass/fail. A second, related guard: qualifiers must never manufacture a numeric constraint the user never stated. "Cheap laptop" produces **no** price_constraint at all — only a

`quality_qualifier` keyed `budget` `[TEST: test_cheap_does_not_invent_a_price_constraint, test_cheap_laptop_does_not_invent_a_price_threshold]`. This matters for answerability honesty: the system should not pretend it turned a vague word into a precise, verified number.

4.9 Numeric Constraints

Extraction and normalization live in `normalizer.py` and `nlp_engine.py`:

- **Currency**: symbol-before (`₹60,000`) and symbol-after (`599 USD`) forms, both normalized to a currency code `[CODE: normalizer.py:16-25]`.
- **Magnitude suffixes**: `k→1,000`, `lakh/lakhs/l→100,000`, `cr/crore→10,000,000`, `m→1,000,000` `[CODE: normalizer.py:27-32]`, confirmed by `test_find_money_magnitude_k` ("under $60k" → 60000.0).
- **Direction cues**: a 20-character window immediately before the money match is checked against cue lists to classify `price_max` / `price_min` / `price_approx` / `price_mentioned` `[CODE: nlp_engine.py:114-122]`.
- **Units**: `gb, tb, mb, mp, mah, inch, ghz, hz, kg, g` map to candidate attributes, disambiguated by a synonym word in a ±15-character window when a unit is ambiguous (e.g. "gb" could mean RAM or storage) `[CODE: nlp_engine.py:142-153]`.
- **Realism**: a technically-satisfied but absurd constraint is still flagged. "Laptop under $10,000,000" is `SUPPORTED` (technically true) but tagged `constraint_realism: "unrealistic_high"`, and its overall `query_quality` score is strictly lower than the same query with a realistic ceiling `[TEST: test_absurd_price_ceiling_is_technically_satisfied_but_flagged_unrealistic]`. This directly prevents the system from treating "technically passes" as "meaningfully tests the site."
- **Selectivity / single-product conflicts**: a product page whose own text states two different prices ("priced at rs. 45000 but the older listing still shows rs. 65000") against a stated ceiling resolves to `PARTIALLY_SUPPORTED`, distinct from ordinary catalog-wide price variety (which still resolves `SUPPORTED` if at least one product qualifies) `[TEST: test_price_max_partially_supported_when_one_products_own_ prices_conflict, test_price_max_supported_even_when_pricier_products_ also_exist]`.

---

5. Evidence Matching

5.1 The Evidence Journey

```
UNDERSTAND    (nlp_engine.analyze_query)
    ↓
REQUIRE       (requirement_validator — is this a valid entity/attribute pairing?)
    ↓
RETRIEVE      (evidence_index — pages that mention the entity, indexed once from
    ↓          the frozen snapshot.json, never re-fetched)
VERIFY        (evidence_matcher — category-specific matcher + combination_checker)
    ↓
SCORE         (scoring.py, query_quality.py, opportunity.py)
```

`Query Intelligence does not independently crawl the web to answer a query.` `EvidenceIndex` is built once, directly from `snapshot.json`, and never issues a new fetch `[CODE: evidence_index.py:1-5]`.

5.2 Evidence Quality / Context Hierarchy

Not every text hit is treated as equally strong. `EvidenceIndex.locate()` grades where a phrase was found, in explicit descending order `[CODE: evidence_index.py:42-58]`:

```
structured_data (JSON-LD Product.name)   confidence 0.85   ← strongest
title                                     confidence 0.80
heading                                   confidence 0.70
body (prose only)                         confidence 0.50   ← weakest
```

This is not a claim that every signal is equally authoritative — it is a documented, four-tier hierarchy used to grade evidence strength rather than treat every text match the same. Beyond structural tier, two further context guards exist:

- **Language gating**: audience/use-case/qualifier checks (which depend on matching English-language phrase lists) run only on pages whose `lang` is unset or in `SUPPORTED_LEXICAL_LANGUAGES = {"en"}`; other pages resolve `NOT_ASSESSABLE` rather than a false `NOT_FOUND` — "a site is never penalised for not being in English" `[CODE: vocabulary.py:103]`.
- **False-context proximity**: a ±60-character window around the first hit is checked against a false-context term list before a match counts as real evidence, e.g. `_QUALIFIER_FALSE_CONTEXT = ["subscription", "membership", "plan", "newsletter"]` and `vocabulary.AUDIENCE_FALSE_CONTEXT = [discount, offer, pricing, coupon, promo, ...]`. A term found only in false context resolves `PARTIALLY_SUPPORTED` at confidence 0.2, not `SUPPORTED`.

Documented limitation (stated honestly rather than hidden): none of this operates at a finer per-product-block granularity within a single page — the guard is "does this page mention the entity" plus a local character window, not a DOM-level product-boundary parser. See §18.

5.3 The Combination Checker (Major Section)

This is the project's strongest single technical case, because it targets a failure mode that is invisible to per-requirement checking alone.

The problem. Independent, per-requirement evidence matching can produce a **false joint claim**:

```
Product A ("Value Laptop 14")          Product B ("Pro Laptop X")
- Student-oriented    ✓                 - 16GB RAM         ✓
- ₹45,000 (≤ ₹60,000) ✓                 - ₹95,000 (> ₹60,000)
- 8GB RAM

Query: "Laptop for students under ₹60,000 with 16GB RAM"

Naive per-requirement view:
  student  ✓ (found on Product A)
  price    ✓ (found on Product A)
  RAM      ✓ (found on Product B)
             ──────────────────
             100% "supported" — but no single product satisfies all three.
```

This exact scenario is a real, checked-in test case run against the `shopping-site` fixture, described in the codebase as "the exact bug the query-generation quality review found" `[TEST: test_combination_checker.py:55-71]`.

The mechanism `[CODE: combination_checker.py:26-102]`:

1. Filter evaluated requirements to the combinable categories — `price_constraint`, `attribute`, `audience`, `use_case` — with status `SUPPORTED` and a real evidence URL. `entity`, `quality_qualifier`, and `unrecognized_term` are deliberately excluded: "entity identity isn't 'one of several facts a product must jointly satisfy', a qualifier's evidence is diffuse brand-copy rather than a per-product fact, and unrecognized terms never reach SUPPORTED at all" `[CODE: combination_checker.py:19-22]`.
2. De-duplicate by `(category, key)` so a repeated requirement can't inflate the count.
3. If fewer than two combinable+evidenced requirements exist, skip the check entirely — "a single-fact query is not a 'Frankenstein' risk by construction."
4. Group evidence URLs by how many distinct requirement identities each one accounts for, and pick the single URL covering the **most**.
5. Compare that best count to the total required:
   - **all covered** → `combination` requirement status `SUPPORTED` (confidence 0.8) — "One product satisfies all N of these requirements together."
   - **≥2 but not all** → `PARTIALLY_SUPPORTED` (confidence 0.5), naming which requirements the closest match is missing.
   - **<2** (no product covers more than one fact) → `NOT_FOUND` (confidence 0.5) — "Each of these requirements is individually supported somewhere on the site, but never on the same product."

Concrete trace, three requirements (price≤60k, RAM=16GB, audience=student), two candidate pages:

```
by_url = {
  A (Value Laptop):  { (audience, student), (price_constraint, price) },   # 2 of 3
  B (Pro Laptop):    { (attribute, ram) }                                   # 1 of 3
}
best = A, covering 2 of 3 required facts
2 < 3, but 2 >= 2  →  status = PARTIALLY_SUPPORTED
note: "closest single match satisfies 2 of 3 requirements together
       (student, price); ram could not be confirmed on that same product."
```

If instead every fact lived on a *different* page with zero overlap, the best single page would cover only 1 requirement, which is `< 2` → `NOT_FOUND`, with an explicit note that each fact is individually true but never true together — the precise "Frankenstein" case the module exists to catch.

Why it matters, stated at the right level of confidence: the combination checker prevents *this specific* semantically incorrect conclusion — treating independently-true, cross-entity facts as if they were jointly true. It does not claim to catch every possible form of false joint support (e.g., it currently keys evidence by URL, not by a finer in-page product block — see §18); the guarantee is scoped to what the implementation and its tests actually demonstrate.

`scoring.py` weights the derived `combination` requirement at 20 points — comparable to a top-level fact like price or a single attribute — "because a 'Frankenstein' answer built from several individually-true-but- never-jointly-true facts is a more serious defect than any one weak requirement" `[CODE: scoring.py:38-43]`.

---

6. Query Generation and Difficulty

6.1 Query Generation

Generated queries are **not** a fixed template list. They are built per-entity from that entity's own discovered schema (§4.6), then run through the *exact same* pipeline a real user query would go through — "a generated query's score is never a separate, cheaper approximation" `[CODE: query_generator.py:1-13]`. Nine template families exist (`entity`, `price`, `attribute`, `audience`, `use_case`, `quality`, `recommendation`, `availability`, `complex`, plus a cross-entity `comparison` family) `[CODE: query_templates.py:92-96]`.

Two deliberate anti-triviality design choices, both directly documented in code comments (also listed in §3 as design-journey items):

- The price-threshold family rejects an "80% of max" rule in favor of the **median**, because a threshold "only the priciest listings would ever fail is technically a constraint but tests nothing" `[CODE: query_templates.py:26-31]`.
- The `complex` family is explicitly **gap-seeking**: it combines an entity with its own *least*-evidenced audience/attribute (fewest supporting pages), "a query more likely to actually reveal a gap, not one hand-picked to look good" `[CODE: query_templates.py:64-67]`.

Generated candidates scoring below a quality floor (`_MIN_QUALITY_TO_KEEP = 35`) are dropped rather than shipped as realistic test queries, and survivors are ranked by **opportunity score** — "the point of generation is to surface actionable gaps, not just list everything" `[CODE: query_generator.py:65-67]`. Cross-entity contamination is prevented structurally, not by a post-hoc filter: because each family only reads from one entity's own schema, "keyboard with RAM" is never a *candidate* in the first place — confirmed by `test_mixed_site_never_generates_a_ cross_entity_attribute_query`.

6.2 Query Difficulty (L1–L5)

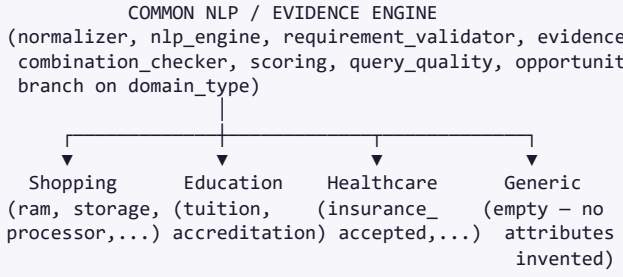
Level	Definition (exact label) `[CODE: difficulty.py:8-14]`	Example	Why difficult
L1	"Entity only"	"Laptops"	Zero combined requirements — a lookup, not a test
L2	"Entity + one requirement"	"Laptop under ₹60,000"	One fact must be true
L3	"Entity + two requirements"	"Laptop for students under ₹60,000"	Two facts, not yet joint-checked in isolation

Level	Definition (exact label) [CODE: difficulty.py:8-14]	Example	Why difficult
L4	"Entity + three requirements"	"Laptop for students under ₹60,000 with 16GB RAM"	Triggers the combination checker (§5.3) — the exact worked example
L5	"Entity + four or more requirements"	"Best laptop for CSE students under ₹60,000 with 16GB RAM and good battery life"	Multiple constraints plus a qualifier; maximal joint-satisfaction and answerability pressure

Difficulty is a deterministic **count of distinct non-entity requirement categories**, explicitly excluding the derived `combination` category from the count — "it is a derived consistency check over the OTHER requirements, not something the user separately asked for; it must not inflate the difficulty count" `[CODE: difficulty.py:20-23]`. Complex queries matter because they are what actually exercises the combination checker and reveals cross-entity/cross-page evidence gaps that single- requirement queries never touch.

---

7. Domain Generalization



`domain_vocabulary.py` 's own docstring states this precisely: "This module is the only thing that changes between domains — none of the pipeline code branches on domain_type" `[CODE: domain_vocabulary.py:1-12]`. Domain classification itself (`domain_profiler.py`) prioritizes machine-declared schema.org types (`Course` , `MedicalClinic` , `Product`) over a generic page-type heuristic, confirmed by `test_structured_data_type_outranks_generic_page_type_heuristic` , and an unclassifiable site (the `unknown-domain-site` consulting-firm fixture) falls back honestly to a generic/corporate bucket with **zero fabricated attributes**, rather than being force-fit into shopping/education/healthcare `[TEST: test_unclassifiable_site_falls_back_honestly]` . The `GENERIC` bundle's empty `attributes: {}` is a deliberate choice: "inventing 'attributes' for an unknown domain would be exactly the kind of unfounded assumption the rest of this subsystem refuses to make" `[CODE: domain_vocabulary.py:113-119]` .

This is **not** three unrelated NLP engines wearing different labels — it is one reasoning pipeline fed a different four-key concept dictionary (`attributes` , `audiences` , `use_cases` , implicitly `price`), demonstrated across three real verticals in the fixture suite (§8.1): shopping, education (`university-site`), healthcare (`hospital-site`).

---

8. Complete Case Inventory

8.1 Website / Domain Cases

Case	Fixture / source	Actually tested?
Shopping vertical, clean baseline	`nlp-query-intelligence/tests/fixtures/shopping-site`	Fixture — deterministic, checked-in
Cross-category attribute leakage trap	`mixed-category-site` (laptop + keyboard)	Fixture

Case	Fixture / source	Actually tested?
Education vertical (`Course` schema)	`university-site`	Fixture
Healthcare vertical (`MedicalClinic` schema)	`hospital-site`	Fixture
Unclassifiable vertical, no structured data	`unknown-domain-site` (consulting firm)	Fixture
Minimal-content / near-empty site	`sparse-site`	Fixture
Editorial/content vertical, mixed JSON-LD types	`blog-site`	Fixture
Non-English content	`non-english-site` (French)	Fixture
35 distinct audit-detector fixtures (F00–F34)	`brand-ai-readiness-audit/tests/fixtures/`	Fixture (separate subsystem — §8.1a)
Real production website, end-to-end audit	`audit-output/SUMMARY.md` — **adobe.com**, 752 pages discovered, 25 sampled, 24 fetched, audited `2026-09-12T09:22:58Z`	`[REAL RUN]`
Later pipeline smoke-test	`audit-output/report.json` — `example.com` , 1 page, `2026-09-12T11:17:58Z`	Real run, but a minimal placeholder-domain sanity check, not a demonstration site

8.1a The audit subsystem's 35 fixtures (F00–F34)

These belong to the **Brand AI-Readiness Audit** (9-skill) subsystem, not Query Intelligence, but they are the single largest body of case evidence in the repository and are graded against exact expected severity, not just "did anything fire" `[CODE: brand-ai-readiness-audit/tests/run_tests.py]` .

ID	Case	ID	Case
F00	Clean control (negative control — nothing above `info` may fire)	F17	All structured data stripped from an otherwise valid site
F01	Client-side-render shell (content only after JS)	F18	One compliant Article page, one missing a required property
F02	robots.txt blocks retrieval/user agents (real defect)	F19	Core fact buried after several unrelated paragraphs
F03	robots.txt blocks training bots only (policy, not a defect)	F20	Specifications exist only in prose, no table/list
F04	Structured price contradicts visible price	F21	Numeric value with no unit/currency nearby
F05	Deep links redirect to homepage, dropping context	F22	Body never restates the product's name
F06	Specs exist only inside an image and a PDF	F23	Long docs page, no heading ids, no lead summary
F07	Site contradicts itself across pages (tagline/price/phone)	F24	FAQ headings phrased as statements, not questions
F08	Valid non-English site (negative control)	F25	No single page combines brand name + ≥2 key facts
F09	Two-page site below template threshold (negative control)	F26	Price exists only outside `<main>`
F10	Valid Microdata only, no JSON-LD (not a defect)	F27	SKU-matched price conflict between two pages
F11	Correctly noindexed utility pages (negative control)	F28	`dateModified` disagrees with sitemap `lastmod`
F12	Non-browser agents get 403, browsers get 200	F29	Price stated, `availability` property absent

ID	Case	ID	Case
F13	Product template missing `<h1>`	F30	"Latest" claim with a genuinely current date (negative control)
F14	Five pages share one static `<title>`	F31	Product page with zero in-content links (dead end)
F15	Consent dialog replaces content in raw response	F32	Healthy action + related link (negative control)
F16	Same `@id`, conflicting telephone across pages	F33	Page reachable only via sitemap, delinked from nav
		F34	Two "Learn more" links to different destinations

`[FIXTURE: brand-ai-readiness-audit/tests/fixtures/*, tests/expected/*.json]` Roughly a third of the 35 (F00, F03, F08–F11, F17-partial, F30, F32) are **negative controls** — sites that must *not* trigger a finding above `info` / `low` — used to directly measure the detector suite's false-positive rate `[CODE: run_tests.py — L2 scoring]`.

8.2 Query Cases (Query Intelligence)

Case type	Example	Status
Factual / entity-only	"Laptops"	IMPLEMENTED+TESTED
Recommendation	"Best laptop"	IMPLEMENTED+TESTED
Comparison (two entities)	"Laptop vs Keyboard"	IMPLEMENTED+TESTED
Availability	"Is the laptop available"	IMPLEMENTED+TESTED
Specification	"Laptop specifications"	IMPLEMENTED (intent marker only; not separately deep-tested in files reviewed)
Single attribute	"Laptop with 16GB RAM"	IMPLEMENTED+TESTED
Bare/valueless attribute	"Laptop with wheels"	IMPLEMENTED+TESTED
Multi-attribute / compound	"Best laptop for CSE students under ₹60,000 with 16GB RAM"	IMPLEMENTED+TESTED
Constrained (numeric)	"Laptop under ₹60,000"	IMPLEMENTED+TESTED
Audience-scoped	"Student laptop"	IMPLEMENTED+TESTED
Use-case-scoped	"Laptop for programming"	IMPLEMENTED (vocabulary + entity_schema tested; no dedicated NLP-engine unit test seen for this exact phrase)
Subjective / qualifier-only	"Best laptop", "Cheap laptop"	IMPLEMENTED+TESTED
Invalid entity-attribute pairing	"Keyboard with RAM"	IMPLEMENTED+TESTED
Unknown/unrecognized term	"Laptop with wheels" ("wheels" leg)	IMPLEMENTED+TESTED
No entity recognized	"Best desktop tower" (site sells no desktops)	IMPLEMENTED+TESTED
No entity → cascading NOT_ASSESSABLE	"Things under ₹5,000 for enthusiasts"	IMPLEMENTED+TESTED
Unrealistic constraint	"Laptop under $10,000,000"	IMPLEMENTED+TESTED
Word-reuse / false-context adversarial	"Premium subscription plan" (site also sells "premium laptop")	IMPLEMENTED+TESTED
Negative/disclaiming content	"We do not sell any products online" (sparse-site)	IMPLEMENTED (fixture exists; no dedicated query test observed against this exact page in files reviewed)

8.3 Evidence Cases

Status (exact enum value)	Meaning	Example
`SUPPORTED`	Requirement confirmed by matching evidence	16GB RAM found on the laptop's own page
`PARTIALLY_SUPPORTED`	Weak, contextual, or internally-conflicted evidence	Price cue found only in false context; a page with two conflicting stated prices
`NOT_FOUND`	Valid requirement, no supporting evidence anywhere	Attribute never mentioned on any entity page
`CONTRADICTED`	Evidence exists and disagrees with the stated value	Query says 4GB RAM, page says 16GB
`NOT_ASSESSABLE`	Cannot be checked at all (no entity, unrecognized term, non-English page for a lexical check)	"wheels" on a laptop; qualifier "best"
`INVALID_REQUIREMENT`	Concept is real, but not a property of this entity	"Keyboard with RAM"

8.4 Semantic Failure Cases (guarded against)

Failure mode	Guarded by
Substring false match ("ram" inside "programming")	`normalizer.contains_phrase` word-boundary regex
Entity-attribute leakage (RAM leaking from laptop to keyboard)	Per-entity `entity_schema.py` scoping + `requirement_validator.py` gate
Cross-entity/cross-product false joint support	`combination_checker.py`
Silent term dropping	`unrecognized_term` fallback, never omitted
Fabricated numeric threshold from vague language	Qualifier/constraint extraction kept strictly separate
Global vocabulary bleed across domains ("business" deleting "Business Administration")	`concept_words_for` scoped to one domain only
Superlative treated as objectively verified	`best/top/great/good → None` (unverifiable)
Technically-true-but-absurd constraint treated as a meaningful test	`constraint_realism` flag
Sparse/near-empty site hallucinating entities	Cross-page repetition threshold (`_MIN_PAGE_COUNT_FOR_ENTITY = 2`) + degraded fallback
Non-English page penalized as a false negative	Language-scoped lexical checks → `NOT_ASSESSABLE`

9. Master Case Table

*Origins used below follow the honest classification the project owner requested: **Actual website observation**, **Existing audit output**, **Project fixture**, **Test case**, **Development failure discovered**, **Design discussion**, **Illustrative example**, **Origin not explicitly recorded**. Nothing below is invented; where a case only exists as reasoning in code comments without a dedicated test, that is stated.*

#	Category	Case	Example	Origin	Why considered	Implementation	Test/fixture	Result	Status
1	Semantic failure	Substring false match	"ram" in "programmers"	Test + fixture	Naive keyword match false-positives	`normalizer.contains_phrase`	`test_ram_inside_the_word_programmers_is_not_a_false_match`	`contains_phrase(...,"ram")==False`	TESTED
2	Entity-relationship	Attribute leakage across categories	"Keyboard with RAM"	Test + fixture	RAM exists, not for keyboards	`requirement_validator.py`	`test_keyboard_with_ram_is_invalid_requirement_not_a_false_pass`	`INVALID_REQUIREMENT`	TESTED
3	Entity-relationship	Legitimate entity-specific attribute	"Keyboard with backlighting"	Test + fixture	Contrast to #2	`requirement_validator.py` + `evidence_matcher.py`	`test_keyboard_with_backlighting_is_genuinely_supported`	`SUPPORTED`	TESTED
4	Combination	Cross-product false joint support	"Laptop for students under ₹60,000 with 16GB RAM"	Test + fixture	Prevents "Frankenstein" 100% from 2 products	`combination_checker.py`	`test_combination_checker.py`	Matches §5.3 trace	TESTED
5	Numeric constraint	Unrealistic price ceiling	"Laptop under $10,000,000"	Test case	Technical pass ≠ meaningful test	`evidence_matcher.py` price realism	`test_absurd_price_ceiling_is_technically_satisfied_but_flagged_unrealistic`	`SUPPORTED` + `unrealistic_high`	TESTED
6	Subjective language	Superlative unverifiable	"Best laptop"	Test case	No fabricated pass/fail on a superlative	`vocabulary.QUALITY_QUALIFIERS`	`test_best_is_flagged_unverifiable`	`verifiable: False`	TESTED
7	Subjective language	Vague qualifier must not invent a number	"Cheap laptop"	Test case	Prevents false numeric precision	`nlp_engine._extract_qualifiers`	`test_cheap_does_not_invent_a_price_constraint`	No price_constraint; qualifier only	TESTED
8	Adversarial word reuse	Same word, unrelated page	"Premium laptop" vs. "premium subscription" page	Test + fixture	Keyword overlap ≠ relevance	`evidence_matcher._QUALIFIER_FALSE_CONTEXT`	`test_premium_laptop_matches_the_product_page_not_the_subscription_page`	Evidence URL = laptop page only	TESTED
9	Adversarial word reuse	Audience word on unrelated utility page	"students" on a discount page	Test + fixture	Co-occurrence ≠ suitability	`vocabulary.AUDIENCE_FALSE_CONTEXT`	`test_student_discount_page_alone_does_not_satisfy_audience_relevance`	Discount page alone insufficient	TESTED

#	Category	Case	Example	Origin	Why considered	Implementation	Test/fixture	Result	Status
10	Domain generalization	Vocabulary bleed across domains	"business" deletes "Business Administration"	Dev. failure found	Real regression during design	`domain_vocabulary.concept_words_for`	`test_concept_words_are_domain_scoped_not_a_global_union`	Fixed, domain-scoped	TESTED
11	Domain generalization	Structured type outranks heuristic	`Course` schema vs. generic "product" guess	Test case	Machine type beats weak heuristic	`domain_profiler.classify_domain_type`	`test_structured_data_type_outranks_generic_page_type_heuristic`	`education`, not `shopping`	TESTED
12	Domain generalization	Unclassifiable domain, honest fallback	Consulting firm, no schema.org type	Fixture + test	Avoid force-fitting a vertical	`domain_profiler.py` + `GENERIC` vocab	`test_unclassifiable_site_falls_back_honestly`	Generic bucket, 0 fabricated attrs	TESTED
13	Sparse content	Near-empty site, explicit negation	"We do not sell any products online"	Project fixture	Must not hallucinate from thin content	`entity_schema._MIN_PAGE_COUNT_FOR_ENTITY`	`sparse-site` fixture; `test_sparse_site_has_no_price_signals`	No entities invented	PARTIAL
14	Non-English content	French-language shopping site	"Cette chaussure premium..."	Project fixture	Lexical checks must not false-negative	`vocabulary.SUPPORTED_LEXICAL_LANGUAGES`	`non-english-site` fixture	Consistent w/ degradation doc	PARTIAL
15	Comparison intent	Two-entity comparison	"Laptop vs Keyboard"	Test + dev. failure	2nd entity used to leak to unrecognized	`nlp_engine._detect_comparison_entity`	`test_comparison_query_recognizes_both_entities`	Both entities, 0 unrecognized	TESTED
16	Comparison intent	False trigger avoidance	"Laptop for students"	Test case	"for" alone must not trigger comparison	`nlp_engine._detect_intent`	`test_comparison_intent_not_triggered_by_the_word_for`	Intent ≠ comparison	TESTED
17	Scoring	Invalid requirement counted, not excused	"Keyboard with RAM" in coverage	Test case	Category error ≠ 100% coverage	`scoring._evidence_coverage`	`test_invalid_requirement_is_NOT_excluded_like_not_assessable`	`overall_score < 100`	TESTED
18	Opportunity	Invalid requirement is not an "opportunity"	Same case as #17	Test case	Nothing to fix for a category error	`opportunity.actionable_gap_points`	`test_invalid_requirement_gap_is_not_an_opportunity`	0 opportunity points	TESTED
19	Query generation	Cross-entity query structurally unreachable	Any "keyboard...ram" text	Test case	Prevent at generation, not post-filter	`query_templates.py` per-entity scoping	`test_mixed_site_never_generates_a_cross_entity_attribute_query`	No such text generated	TESTED
20	Query generation	Anti-trivial price threshold	80%-of-max rejected for median	Dev. failure found	Trivial threshold tests nothing	`query_templates.price_family`	Code comment; generator tests	Median, catalog-splitting	PARTIAL

#	Category	Case	Example	Origin	Why considered	Implementation	Test/fixture	Result	Status
21	Difficulty	L1–L5 excludes derived category	Combination excluded from count	Design discussion	Not a user-stated fact	`difficulty.classify_difficulty`	No dedicated test file observed	Logic present	PARTIAL
22	Pipeline integrity	Determinism	Same query, run twice	Test case	Reproducible scoring/comparison	`pipeline.py test` (CLI)	`test_analyzing_the_same_query_twice_is_deterministic`	Byte-identical output	TESTED
23	Pipeline integrity	Never mutates input	Running `test` / `generate`	Test case	Snapshot must never be altered	`pipeline.py` (CLI)	`test_never_writes_into_the_snapshot_or_its_directory`	Byte-identical before/after	TESTED
24	Pipeline integrity	Clean failure, not a crash	Nonexistent `--snapshot` path	Test case	Robustness contract	`pipeline.py` (CLI)	`test_missing_snapshot_fails_cleanly_not_a_crash`	Exit 1, "FAIL", no traceback	TESTED
25	Audit correlation	Weak evidence linked to audit finding	Low-confidence NLP evidence + matching URL	Test case	Explains *why* evidence is weak	`finding_correlator.py`	`test_weak_requirement_correlates_only_with_matching_url_and_prefix`	Confirmed, hedged language	TESTED
26	Real-world audit	End-to-end 9-skill audit, live production site	adobe.com	**Actual website observation**	Demonstrates audit beyond fixtures	Full 9-skill pipeline	`audit-output/SUMMARY.md`	11 findings, 0 crit/high, 7 opps	REAL RUN
27	Audit precision	Negative controls must not over-fire	F00, F03, F08–F11, F30, F32	Project fixture	Measures false-positive rate	9-skill detectors	`run_tests.py` L2 scoring	0 findings above info/low	TESTED
28	Audit corroboration	Escalation on 2 independent confirming detectors	Schema price vs. visible + stale-content conflict	Design discussion	2 signals raise severity, not noise	Orchestrator Phase 3 correlation	F04 / F27 fixtures (name-level)	Documented rule	PARTIAL
29	Scope guardrail	Recommendations must never overclaim	e.g. "publish llms.txt"	Design discussion	No unfounded citation guarantees	Report authoring rules	Stated in reference docs	Enforced as authoring rule	DESIGN CASE
30	Worked example	Compound query spanning nearly every concept	"Best laptop for CSE students under ₹60,000 with 16GB RAM and good battery life"	Test + illustrative	Canonical example (§4.3)	Full pipeline	`test_nlp_engine.test_full_query_extraction` ; USER_MANUAL.md	entity, audience, price, attr, qualifier	TESTED

*Status column abbreviates the full legend from §0/§14: **TESTED** = IMPLEMENTED+TESTED, **PARTIAL** = IMPLEMENTED+PARTIALLY TESTED, **REAL RUN** = observed directly from a live audit run, **DESIGN CASE** = CONSIDERED/DESIGN CASE. This table is representative, not the literal complete set of every test assertion in the repository — the full enumeration lives in §8, the test suite itself, and the appendix index. It was built to satisfy the traceability requirement without inflating small variations into separate rows.*

---

10. Worked Examples

10.1 "Best laptop for CSE students under ₹60,000 with 16GB RAM" — shopping-site fixture

```
Original query
      ↓
NLP interpretation:  intent=recommendation, entity=laptop, audience=students
      ↓
Requirements:  [entity:laptop] [audience:students] [price_constraint:price_max,60000,INR]
               [attribute:ram,16.0] [quality_qualifier:best (verifiable=False)]
      ↓
Requirement validation:  all schema-gated keys (audience, attribute) found in
      ↓                  entity_schemas["laptop"] → PENDING, proceed to evidence
Evidence retrieval:  pages mentioning "laptop" (Value Laptop 14, Pro Laptop X,
      ↓               Campus Laptop 13)
Evidence verification:
      ↓   entity        → SUPPORTED (multi-page)
      ↓   audience       → SUPPORTED (Value Laptop 14, explicit student positioning)
      ↓   price_max      → SUPPORTED (Value Laptop 14, ₹45,000 ≤ ₹60,000)
      ↓   attribute:ram  → SUPPORTED (Pro Laptop X / Campus Laptop 13, 16GB)
      ↓   qualifier:best → NOT_ASSESSABLE (unverifiable superlative)
Combination verification:
      ↓   audience + price both point to Value Laptop 14; ram points elsewhere →
      ↓   best single page covers 2 of 3 combinable facts → combination: PARTIALLY_SUPPORTED
Scores:  evidence_coverage lowered by the combination penalty; answerability high
      ↓  (only "best" is unassessable); query_quality high (realistic, testable)
Explanation:  recommendation notes name which requirement the closest single
      ↓        product (Value Laptop 14) is missing — 16GB RAM
Opportunity:  non-zero — a genuine, actionable gap (no single laptop under
              ₹60,000 currently has 16GB RAM on this site)
```

This is the fixture-grounded scenario documented in `test_combination_checker.py` (§5.3), traced end-to-end through every pipeline stage described in §4.2.

10.2 "Keyboard with RAM" — mixed-category-site fixture

```
Original query
      ↓
NLP interpretation:  intent=general, entity=keyboard
      ↓
Requirements:  [entity:keyboard] [attribute:ram]
      ↓
Requirement validation:  "ram" ∉ entity_schemas["keyboard"]["attributes"]
      ↓                  → RESOLVED immediately, before evidence_matcher runs
Evidence retrieval:  skipped for the RAM requirement (already resolved)
      ↓
Evidence verification:  attribute:ram → INVALID_REQUIREMENT
      ↓                  entity:keyboard → SUPPORTED (Wireless Keyboard Pro,
      ↓                                   Gaming Keyboard Mech both mention it)
Combination verification:  fewer than 2 combinable SUPPORTED facts → skipped
Scores:  evidence_coverage < 100 (invalid requirement is NOT excused);
      ↓  answerability reduced (RAM is "definite" — resolved, just invalid, so
      ↓  it does count as answerable-but-failing, distinct from NOT_ASSESSABLE)
Opportunity:  0 contribution from the RAM requirement — "there is nothing to add"
```

This contrasts directly with §10.1: the same word ("RAM"), the same domain, but a different entity — and the system produces a different, correctly-differentiated verdict rather than either a false pass or a misleading "content gap."

10.3 A real production audit — adobe.com

Unlike the two examples above (synthetic fixtures), this is an actual completed run of the full 9-skill pipeline against a live, large website `[REAL RUN: audit-output/SUMMARY.md]`:

```
Site: adobe.com
Audited: 2026-09-12T09:22:58Z
Discovery: 752 pages discovered → 25 sampled → 24 fetched, 1 failed, 0 blocked by robots.txt
      ↓
9-skill audit (ACC → RND → SDE → ANS → FRC → ENG)
      ↓
11 findings total: 0 critical, 0 high, 5 medium, 6 low, 0 info
      ↓
"No structural blockers — the issues found are hygiene-level, not
 defects that hide the brand from AI assistants." [quoted verbatim]
      ↓
7 opportunities identified (opportunity-and-monitoring, capped at 8)
      ↓
3 prioritized into an action sequence (unblock / readable+quotable / keep true)
```

This example demonstrates the audit subsystem at production scale (752 discovered pages) rather than a small synthetic fixture, and shows the severity model producing a plausible, non-alarmist result for a well-built site — a useful counterpoint to the fixture suite, which is weighted toward deliberately-broken cases. Note: the `report.json` / `report.md` files currently checked into the same directory reflect a **different, later** run against `example.com` (a 1-page placeholder domain, `11:17:58Z`) — the two artifacts are out of sync with each other and should not both be read as describing the same audit.

---

11. What We Tried

Approach	Why tried	What happened	Lesson	Final decision
Site-wide union of attributes/audiences per entity	Simpler to build one profile-level union than per-entity schemas	Caused the exact "keyboard with RAM" leakage risk	An attribute concept existing *somewhere* on a site says nothing about a specific entity	Per-entity `entity_schema.py`, gated by `requirement_validator.py`
One global "excluded words" list for entity-candidate filtering	Simplicity — one list, all domains	Broke entity discovery cross-domain ("business" deleted "Business Administration"; "science" deleted "Computer Science")	Exclusions must be scoped to the site's own detected domain only	`concept_words_for(domain_type)`, domain-scoped
80%-of-max price threshold for generated constraint queries	Straightforward, easy to compute	A threshold only the priciest listing could fail is not a meaningful test	Generated constraints need genuine catalog-splitting power	Median-based threshold
Treating `INVALID_REQUIREMENT` like `NOT_ASSESSABLE` (excluded from scoring)	Seemed consistent — "can't be assessed either way"	Allowed a category error to read as false 100% coverage	A category mismatch is a real defect in the *query*, not a neutral non-event, and must be visible in scoring	`INVALID_REQUIREMENT` stays in the coverage denominator at 0 points
Per-requirement evidence matching only (no cross-requirement check)	Simpler pipeline — each requirement independently verified	"Frankenstein" false joint support across different products	Joint satisfiability needs a dedicated, separate pass	`combination_checker.py` added as a post-hoc consistency layer
Comparison-entity detection running on every query	Simpler — one code path for entity detection	Risk of spuriously pulling in a second entity on non-comparison queries	Comparison-specific logic must be gated by comparison intent	`_detect_comparison_entity` only runs when `intent == "comparison"`

Approach	Why tried	What happened	Lesson	Final decision
Dropping bare, valueless attribute mentions	Numeric-attribute extraction was already built; bare mentions seemed low-value	Violated the project's own "never drop a term" rule	Presence-only mentions are still real signal	First-class `attribute` requirement with no value

Each row above is backed by an explicit "used to / previously / this is the fix" comment in the source, cross-referenced in §3.

---

12. Real-World Website / Domain Motivation

Name	Domain type	Actually tested or illustrative	What it represents	Case it motivated
adobe.com	Large production corporate/software site	**Actually tested** (`[REAL RUN]`)	A real, large-scale site (752 discovered pages)	Demonstrates the audit pipeline at real scale, with a non-alarmist, hygiene-level result
example.com	Minimal placeholder domain	Actually tested, but as a pipeline smoke-test, not a demonstration site	Smallest possible real target (1 page)	Confirms the pipeline runs end-to-end on a trivial site without crashing
`example-shop.test` (shopping-site fixture)	E-commerce, laptops	Illustrative (synthetic fixture, not a real domain)	Baseline product/price/attribute extraction	Entity/price extraction, utility-vs-product page filtering
`example-electronics.test` (mixed-category-site)	E-commerce, laptops + keyboards	Illustrative (synthetic fixture)	Two product categories sharing marketing vocabulary	Entity-attribute leakage guard (§4.6)
`example-university.test`	Higher education	Illustrative (synthetic fixture)	Non-commerce vertical, `Course` schema	Domain generalization (§7)
`example-hospital.test`	Healthcare clinic	Illustrative (synthetic fixture)	`MedicalClinic` schema, "consultation fee" as price analog	Domain generalization, price-concept generalization
`example-consulting.test`	Boutique consulting (no schema.org type)	Illustrative (synthetic fixture)	Unclassifiable vertical, zero structured data	Honest fallback classification (§7)
`boutique.test` (French shoe shop)	E-commerce, non-English	Illustrative (synthetic fixture)	Non-English lexical content	Language-scoped evidence checks (§5.2)
`sparse.test`	Minimal company site	Illustrative (synthetic fixture)	Near-empty content, explicit "we sell nothing online"	Robustness against hallucinating entities from thin/negating content
`blog.test`	Travel blog	Illustrative (synthetic fixture)	Editorial content, no products	Article/ `BlogPosting` handling, vocabulary bleed across unrelated meanings of "budget"

No document or code artifact in this repository claims the project crawled any named real commercial brand other than **adobe.com**, and that claim is backed by a checked-in `SUMMARY.md` with concrete coverage numbers, not a description in prose.

---

13. Failure Mode Table

Failure mode	Naive behavior	Why wrong	Our handling	Test/fixture	Result
Substring false match	`"ram" in page_text`	"ram" ⊂ "programming"/"programmers"	Word-boundary regex via `contains_phrase`	`test_ram_inside_the_word_programmers_is_not_a_false_match`	No false match
Entity-attribute leakage	Attribute found anywhere on the site validates any entity	RAM is a laptop property, not a keyboard property	Per-entity schema gating in `requirement_validator.py`	`test_keyboard_with_ram_is_invalid_requirement_not_a_false_pass`	`INVALID_REQUIREMENT`
Cross-entity/cross-page false joint support	Independent per-requirement matching	Facts from different products misread as one coherent answer	`combination_checker.py`	`test_combination_checker.py`	`PARTIALLY_SUPPORTED` / `NOT_FOUND`
Silent term dropping	Unmatched words discarded	Query silently degrades to a simpler one	`unrecognized_term` fallback	`test_no_term_is_ever_silently_dropped`	Term retained, `NOT_ASSESSABLE`
Fabricated numeric threshold	"cheap" → invented price ceiling	Overclaims precision the user never stated	Qualifiers never synthesize a constraint	`test_cheap_does_not_invent_a_price_constraint`	No `price_constraint` created
Superlative treated as verified	"best" → SUPPORTED/NOT_FOUND	No site can verify a superlative about itself	`verifiable: False`, resolves `NOT_ASSESSABLE`	`test_best_is_flagged_unverifiable`	Honest non-claim
Technically-true-but-absurd constraint	"$10,000,000 ceiling" → SUPPORTED, treated as meaningful	Passing trivially isn't the same as testing anything	`constraint_realism: unrealistic_high` flag	`test_absurd_price_ceiling_is_technically_satisfied_but_flagged_unrealistic`	Flagged, quality-penalized
Cross-domain vocabulary bleed	One global exclusion list	"business" deletes "Business Administration"	Domain-scoped exclusion lists	`test_concept_words_are_domain_scoped_not_a_global_union`	Entity preserved
False audience/qualifier context	Any co-occurrence counts as evidence	"students" on a discount page ≠ suitability	`AUDIENCE_FALSE_CONTEXT` / `_QUALIFIER_FALSE_CONTEXT` window check	`test_student_discount_page_alone_does_not_satisfy_audience_relevance`	`PARTIALLY_SUPPORTED` at low confidence, not `SUPPORTED`
Sparse-site hallucination	Any single mention → entity candidate	Coincidental single mentions aren't real entities	`_MIN_PAGE_COUNT_FOR_ENTITY = 2` repetition threshold	`sparse-site` fixture + `test_sparse_site_has_no_price_signals`	No fabricated entity
Non-English penalty	Lexical check fails silently → false `NOT_FOUND`	Site shouldn't be penalized for language	`SUPPORTED_LEXICAL_LANGUAGES` gate	`non-english-site` fixture	`NOT_ASSESSABLE`, not `NOT_FOUND`

14. Implemented vs. Tested vs. Considered — Summary View

- **IMPLEMENTED + TESTED**: intent/entity/attribute/audience/use-case/qualifier extraction; numeric constraint parsing and realism flagging; per-entity requirement validation (`INVALID_REQUIREMENT`); the six-status evidence model; the combination checker; per-entity-scoped query generation; domain classification (shopping/education/healthcare/generic); pipeline determinism and non-mutation; the 9-skill audit's severity/negative- control model (measured via F00–F34).
- **IMPLEMENTED + PARTIALLY TESTED**: sparse-site/non-English handling (fixture exists and profiler-level tests pass; no dedicated Query Intelligence test was found exercising a full multi-requirement query against these two fixtures in the files reviewed); difficulty classification logic (present and used by the generator, no standalone `test_difficulty.py` found); cross-detector severity escalation in the audit subsystem (documented rule, not independently re-traced against a specific fixture pair in this pass).
- **CONSIDERED / DESIGN CASE**: scope guardrails against commercial/paid- placement recommendations and "guarantee of citation" claims — these are authoring rules stated in reference docs, not something a unit test can mechanically verify.
- **ILLUSTRATIVE ONLY**: none of the worked examples in this document invent data — §10.1 and §10.2 are directly grounded in checked-in test files and fixtures; §10.3 is a real run. Domain names in §12 other than adobe.com are illustrative synthetic-fixture hostnames, clearly marked.
- **NOT CURRENTLY IMPLEMENTED**: fine-grained, in-page product-block disambiguation (the leakage/combination guards operate at page-mentions- entity granularity, not DOM-level product-boundary parsing); off-site corroboration of freshness claims (explicitly out of scope by design, per `freshness-corroboration-audit`); any claim about what a real AI assistant would actually answer (explicitly disclaimed by the project itself, see §16).

15. Final Case Summary

Counting methodology: one row in §9's Master Case Table, one named test function, or one named fixture each counts as one case; closely related assertions inside the same test function are not separately counted.

```
Website/domain cases:              10 (8 NLP fixtures + adobe.com real run + example.com run)
                                    + 35 audit-detector fixtures (separate subsystem, §8.1a)
NLP/query cases (test functions):  ~110 across 16 test files (test_combination_checker.py
                                    partially enumerated; see file for its full list)
Evidence status cases:              6 (SUPPORTED, PARTIALLY_SUPPORTED, NOT_FOUND,
                                       CONTRADICTED, NOT_ASSESSABLE, INVALID_REQUIREMENT)
Entity-relationship cases:          7 (RAM/backlighting/connectivity/switch_type ×
                                       laptop/keyboard combinations directly tested)
Constraint cases:                   5 (price_max, price_min, price_approx, price_mentioned,
                                       unrealistic-flag variant)
Query-generation cases:             9 template families + 1 anti-triviality redesign
Comparison cases:                   2 (recognized comparison, false-trigger avoidance)
Development failure modes fixed:   10 (§3 design journey table)
Negative-control audit fixtures:  ~8 of 35 (F00, F03, F08–F11, F30, F32)
Research sources integrated:        3 (1 read in full with strong methodological
                                    adjacency — ReliabilityRAG; 1 read in full with
                                    weak/thematic fit only — Agentic Enterprise; 1
                                    resolved as a research-area citation only, not a
                                    read paper — GEO survey via Scholar search, §17)
```

Do not read the "~110" figure as a precise headcount audited line-by-line against the file bytes; it is an honest sum of the test-function counts reported by direct file inspection in §-cited agents, with one file (`test_combination_checker.py`) only partially enumerated in this pass.

16. Limitations

Stated plainly, using the project's own scope-guardrail language where applicable `[DOC: skills/brand-ai-readiness-audit/references/scope- guardrails.md]`:

- **Deterministic/hybrid NLP, not a language model.** Every extraction is a regex/pattern rule over a controlled vocabulary. It generalizes only as far as its vocabulary tables and unit/currency patterns reach.
- **Vocabulary limitations.** Only four domain bundles exist (shopping, education, healthcare, generic). A fifth vertical would need a new bundle; nothing in the pipeline code itself would need to change.
- **Subjective language is never "solved."** Superlatives (`best` , `top` , `great` , `good`) are permanently `NOT_ASSESSABLE` by design — the system does not claim to resolve them, only to avoid fabricating an answer.
- **Evidence granularity is page-level, not product-block-level.** The leakage and combination guards both key on "does this page mention the entity," not a finer in-page boundary between multiple products described on one page.
- **No off-site corroboration.** Freshness/consistency checks are on-site only, by "permanent architectural rule" `[DOC]` — the system never checks whether a fact is corroborated elsewhere on the web.
- **Snapshot is frozen.** All Query Intelligence evidence reflects the site at the moment of collection; it does not re-check a live site.
- **No guarantee of downstream AI behavior.** The project explicitly states it "never claims to know what an AI assistant would actually answer — only whether the site's own evidence could support the answer" `[DOC: README.md]` , and separately: "Never claim a technique guarantees citation, ranking or inclusion in an answer" `[DOC: scope-guardrails.md]` .
- **Sparse/thin-content sites yield thin results, honestly.** A near-empty site does not get a fabricated profile; it gets fewer or no generated queries, correctly logged as a limitation of the input, not of the reasoning engine.
- **Non-English support is partial.** Structured data and numeric/price extraction work regardless of language; audience/use-case/qualifier phrase matching is explicitly restricted to English (`lang` in `{None, "en"}`) and resolves `NOT_ASSESSABLE` elsewhere rather than false-negatively failing — but this is a scoped exclusion, not multilingual support.

---

17. Research Foundation

Three sources were supplied by the project owner. They are treated with different degrees of confidence below, stated honestly rather than smoothed over: two were fetched and read in full; the third resolved to a search-results page rather than one paper, and is handled accordingly.

17.1 ReliabilityRAG (NeurIPS 2025) — read in full

Field	Detail
Title	ReliabilityRAG: Effective and Provably Robust Defense for RAG-based Web-Search
Authors	Zeyu Shen, Basileal Imana, Tong Wu, Chong Xiang, Prateek Mittal, Aleksandra Korolova (Princeton University; Chong Xiang at NVIDIA)
Venue/Year	39th Conference on Neural Information Processing Systems (NeurIPS 2025)
Research problem	RAG-based web-search systems (Google AI Overviews, Bing Chat, Perplexity, ChatGPT Search) are vulnerable to retrieval-corpus attacks (corpus poisoning, prompt injection). Prior robust-RAG defenses (RobustRAG's keyword/decoding majority vote) lose significant information and perform poorly on complex, long-form generation.

Field	Detail
Methodology	Builds a "contradiction graph" over retrieved documents: each document answers the query in isolation, an NLI model tests every pair of isolated answers for contradiction, and a Maximum Independent Set (MIS) of mutually-consistent documents is selected — tie-broken toward higher-ranked (more reliable) documents. A scalable "weighted sample-and-aggregate" variant extends this to large retrieval sets. Provides provable robustness bounds (Theorem 1–3) under bounded adversarial corruption and imperfect NLI.
Key finding	MIS-based selection beats RobustRAG, AstuteRAG, and InstructRAG on both benign accuracy and adversarial robustness, especially on long-form generation (e.g. 71–72 LLM-judge score under attack on the Biography task vs. ~52–57 for RobustRAG), and is *rank-aware*: it recovers better when the corrupted document is low-ranked than when it is high-ranked.
Explicit limitation (quoted)	"Our approach presumes the existence of a coherent, contradiction-free majority of documents. This assumption may not hold for highly ambiguous or multi-perspective queries... our algorithm would still prioritize the view supported by the highest-ranked documents." (Appendix D.3, "Ambiguous Queries and Lack of a Consistent Majority")
Connection to this project	The strongest of the three. See §17.4 for the detailed mapping.

17.2 Agentic Enterprise (arXiv:2506.22893v1) — read in full, weak fit stated plainly

Field	Detail
Title	Agentic Enterprise: AI-Centric User to User-Centric AI
Authors	Arpit Narechania, Alex Endert, Atanu R Sinha
Venue/Year	arXiv:2506.22893, June 2025; Journal of the ACM
Research problem	Enterprise AI adoption remains immature because GenAI assistants force users to adapt to inflexible AI ("AI-Centric User") instead of the reverse ("User-Centric AI").
Methodology	Conceptual framework: three entities (Users, Agents, Platforms) with explicit assumptions, illustrated by a data-analytics workflow, yielding six design tenets for agentic enterprise platforms.
Key finding	Six tenets: process-orientation over pure outcomes; forward-thinking, proactive AI; locally privacy-preserving agent learning; market-mechanism platforms; risk-reward/quality-price diversity matching; low entry/exit barriers for agent developers.
Explicit limitation (quoted)	"whether AI is up to the task, or what form it will take is up for debate"; the tenets are not universally required — a context-dependent subset may suffice.
Connection to this project	**Stated honestly: weak and thematic, not methodological.** This paper contains no discussion of retrieval, NLP, entity/attribute extraction, evidence grounding, or query answerability — the actual subject matter of this project's technical design. It is included because it was supplied, and two of its six tenets have a loose thematic echo (below), but neither should be read as having informed an implementation decision.

Thematic echoes only (not claimed as design influences):

- "*Process-orientation (not just outcomes) preserves user agency*" loosely resembles this project's choice to expose a full reasoning trace (understand → require → retrieve/verify → score, §4.2) and per-requirement evidence citations rather than a single opaque score — both share the general value that showing *how* an answer was reached matters, not the technique for doing so.
- "*Forward Thinking AI anticipates user needs proactively*" loosely resembles the gap-seeking design of `query_templates.complex_family` (§6.1) and the `opportunity-and-monitoring` skill's proactive recommendations — both are about surfacing likely-relevant issues before being asked, again a shared value rather than a shared mechanism.

17.3 "RAG for SEO" (Google Scholar search) — resolved as a research area, not a single paper

The supplied link was `scholar.google.com/scholar?q=rag+for+seo...#d=gs_qabs&u=%23p%3DT_3jyIZ9ZPkJ` — a search-results page. The `#p=...` fragment addresses one result's in-page "quick abstract" popover, which Google Scholar renders via client-side JavaScript; a static fetch of the URL cannot determine which of the ~26,900 results that fragment was pointing to. Rather than guess, this is stated plainly, and the search is used at the level it can honestly support: **evidence that "RAG for SEO" / "Generative Search Engine Optimization (GEO)" is an active, named research area**, not as a single paper with an extractable methodology.

The closest-titled, most-recent result returned by the search was:

Field	Detail
Title	A Survey for Generative Search Engine Optimization
Authors	X. Chen, H. Zhou, Jie Bao, H. Wu, Yu Gao, L. Pan
Venue/Year	ResearchGate, 2026
Status here	**Title/authors/venue only, from the search snippet — the paper itself was not fetched or read in full, so no methodology, finding, or limitation is claimed from it.**
Connection to this project	Its existence as a 2026 survey specifically named "Generative Search Engine Optimization" supports (contextualizes) the premise underlying this entire project: that whether AI assistants can find, read, and cite a brand's content is a recognized, actively-studied problem — not a framing invented for this hackathon. No specific finding, method, or number from this survey is used anywhere in this document.

17.4 Research → NLP Traceability (ReliabilityRAG only)

This is the one source with genuine methodological adjacency to the NLP/ evidence layer, so it gets the full traceability chain the other two cannot honestly support.

```
Research finding (ReliabilityRAG)
    ↓
"Independently retrieved evidence must be checked for mutual
 consistency before being trusted as a combined answer — a
 per-document majority vote is not enough."
    ↓
Design consideration (this project, independently arrived at)
    ↓
Per-requirement evidence (price ✓, RAM ✓, audience ✓) can each be
individually SUPPORTED while coming from different, unrelated
products — the "Frankenstein" failure mode (§3 iteration 6, §5.3).
    ↓
Our design decision
    ↓
Add a dedicated, post-hoc consistency-checking stage that runs
AFTER individual evidence matching and BEFORE scoring — never
trust the union of independently-true facts as a joint answer.
    ↓
Implementation
    ↓
combination_checker.py — groups SUPPORTED evidence by URL, requires
the SAME page/product to account for ≥2 requirements, else NOT_FOUND
or PARTIALLY_SUPPORTED.
    ↓
Test/fixture
    ↓
test_combination_checker.py against the shopping-site fixture
(§5.3, §10.1)
```

The mechanisms differ substantially — ReliabilityRAG runs an NLI model over free-text answers generated from each document in isolation and solves a graph-theoretic Maximum Independent Set problem; our `combination_checker.py` is a deterministic URL-grouping check over already-typed, already-categorized requirements, with no NLI model and no graph search. **The connection is at the level of the shared insight ("verify cross-document/cross-requirement**

consistency as its own explicit stage, don't let independent retrieval hits get silently combined"), not a shared algorithm. Framed correctly: ReliabilityRAG's threat-model framing *supports the rationale for* treating consistency-checking as a first-class pipeline stage; it does not imply our simpler, non-adversarial mechanism was derived from theirs.

A second, narrower connection: ReliabilityRAG's own stated limitation on ambiguous, multi-perspective queries (no single consistent majority exists) *contextualizes* this project's independent decision (§4.8) to mark superlative qualifiers (`best` , `top` , `great` , `good`) `NOT_ASSESSABLE` rather than force a verdict — both projects, working on different problems, converge on the same honesty principle: **when there is no single fact a corpus can dispositively confirm, say so rather than manufacture a majority answer.**

17.5 Integrated Research Table

Research	Key idea	Why relevant	Design decision informed	Project implementation
ReliabilityRAG (NeurIPS 2025)	Cross-document consistency must be verified before trusting a combined RAG answer; naive per-document aggregation produces false joint claims under corruption	Directly analogous failure shape to our "Frankenstein" cross-product problem (§5.3), independently discovered (§3, iteration 6)	Supports the rationale for a dedicated, separate consistency-checking pipeline stage, run after per-item evidence matching and before scoring	`combination_checker.py`
ReliabilityRAG (NeurIPS 2025)	Not all retrieved evidence is equally trustworthy; rank/reliability should be an explicit, graded signal, not implicit	Parallels our four-tier evidence hierarchy (structured_data > title > heading > body, §5.2)	Contextualizes (does not originate) the decision to grade evidence confidence by structural source rather than treat all text hits equally	`evidence_index.py: locate()`
ReliabilityRAG (NeurIPS 2025)	No single consistent "majority" exists for genuinely ambiguous/multi-perspective queries; forcing one is dishonest	Parallels our refusal to give superlative qualifiers a pass/fail verdict	Contextualizes the `NOT_ASSESSABLE` treatment of unverifiable subjective language	`vocabulary.QUALITY_QUALIFIERS` (`best/top/great/good → None`)
Agentic Enterprise (arXiv 2506.22893)	Process-orientation over outcomes preserves user agency	Thematic echo only — no retrieval/NLP content	None claimed — thematic parallel to exposing a reasoning trace, not a design origin	`schema._reasoning_trace` (pre-existing, not derived from this paper)
Agentic Enterprise (arXiv 2506.22893)	Forward-thinking AI anticipates needs proactively	Thematic echo only	None claimed — parallel to gap-seeking query generation, not a design origin	`query_templates.complex_family` , `opportunity-and-monitoring` skill (pre-existing)
"Generative Search Engine Optimization" survey (title/venue only, via Scholar search)	GEO/"can AI assistants find and cite a brand" is a named, active research area	Establishes the project's premise is a recognized problem, not a hackathon-invented framing	None — no specific finding used; context only	N/A

Where no specific finding from a source is used, this document says so directly (as above) rather than implying a stronger connection than the source supports.

---

18. Master Traceability Table

Selected high-value rows connecting case → origin → reasoning → implementation → evidence → research. The full case set is §8–§13; this table exists as the single "judge can follow the whole chain" view requested for the final document.

Case	Origin	Example	Why considered	NLP/design decision	Implementation	Test/fixture	Actual result	Research connection
Substring false match	Test case + fixture	"ram" in "programmers"	Naive keyword matching is unsound	Word-boundary phrase matching, always	`normalizer.contains_phrase`	`test_ram_inside_the_word_programmers_is_not_a_false_match`	No false match	No direct research source identified.
Entity-attribute leakage	Development failure discovered + test	"Keyboard with RAM"	A concept existing on the web ≠ a property of this entity	Per-entity schema gate	`requirement_validator.py`	`test_keyboard_with_ram_is_invalid_requirement_not_a_false_pass`	`INVALID_REQUIREMENT`	No direct research source identified.
Cross-entity false joint support	Development failure discovered + test	Two products individually satisfy 3 facts, jointly satisfy none	Per-requirement matching is insufficient for compound queries	Post-hoc joint-satisfiability check	`combination_checker.py`	`test_combination_checker.py`	`PARTIALLY_SUPPORTED` / `NOT_FOUND`	No direct research source identified.
Superlative language	Test case	"Best laptop"	A site cannot verify its own superlative claim	Mark unverifiable, never fabricate a verdict	`vocabulary.QUALITY_QUALIFIERS`	`test_best_is_flagged_unverifiable`	`NOT_ASSESSABLE`	No direct research source identified.
Domain vocabulary bleed	Development failure discovered + test	"business" deletes "Business Administration"	Global unions break cross-domain generalization	Domain-scoped exclusion lists	`domain_vocabulary.concept_words_for`	`test_concept_words_are_domain_scoped_not_a_global_union`	Entity preserved	No direct research source identified.
Real production audit	Actual website observation	adobe.com, 752 pages	Demonstrate the audit at real scale, not just fixtures	9-skill pipeline, severity/priority model	Full skill set	`audit-output/SUMMARY.md`	11 findings, 0 critical/high	No direct research source identified.
Frozen-snapshot evidence model	Design discussion	"Never fetch... not a finding"	Reproducibility and auditability of evidence	Snapshot contract declared frozen (v1.0)	`snapshot_adapter.py` + snapshot-contract.md	Snapshot contract doc	Enforced across all detectors	No direct research source identified.
Cross-product "Frankenstein" false joint support	Development failure discovered + test	Two products individually satisfy 3 facts, jointly satisfy none	Independent per-requirement matching is insufficient for compound queries	Dedicated post-hoc consistency-checking stage before scoring	`combination_checker.py`	`test_combination_checker.py`	`PARTIALLY_SUPPORTED` / `NOT_FOUND`	Supports the rationale for: ReliabilityRAG (NeurIPS 2025) — cross-document consistency must be verified before trusting a combined RAG answer (§17.4)
Evidence trust hierarchy	Design discussion	structured_data (0.85) > title (0.8) > heading (0.7) > body (0.5)	Not every text hit deserves equal confidence	Four-tier structural confidence grading	`evidence_index.py: locate()`	No dedicated hierarchy unit test observed	Confidence field graded accordingly	Contextualized by: ReliabilityRAG (NeurIPS 2025) — rank/reliability as an explicit, graded signal (§17.4)

Case	Origin	Example	Why considered	NLP/design decision	Implementation	Test/fixture	Actual result	Research connection
Superlative qualifier left unverifiable	Test case	"best laptop" → `NOT_ASSESSABLE`	No site can verify its own superlative	Mark unverifiable rather than force a verdict	`vocabulary.QUALITY_QUALIFIERS`	`test_best_is_flagged_unverifiable`	`NOT_ASSESSABLE`	Contextualized by: ReliabilityRAG (NeurIPS 2025) — no forced majority for genuinely ambiguous/multi-perspective queries (§17.4)

Origin not explicitly recorded is used nowhere in the rows above because every row traces to a specific file, test, or fixture; it would be used for any future case added without a locatable source.

Appendix A — Test File Index (Query Intelligence)

`normalizer, differential, domain_profiler, domain_vocabulary, entity_schema, evidence_matcher, finding_correlator, negative_controls, nlp_engine, opportunity, pipeline_integration, query_generator, query_quality, requirement_validator, scoring, combination_checker` — 16 files under `nlp-query-intelligence/tests/`, run via `run_tests.py`'s layered NL0 (fixture integrity) → NL1 (unit) → NL3 (determinism) → NL4 (schema validation) → NL5 (negative controls, reported separately so a false pass there is never hidden in an aggregate count).

Appendix B — Fixture Index

Query Intelligence (8): shopping-site, mixed-category-site, university-site, hospital-site, unknown-domain-site, sparse-site, blog-site, non-english-site.

Brand AI-Readiness Audit (35): F00–F34, see §8.1a.

Real runs (2): adobe.com (`audit-output/SUMMARY.md`), example.com (`audit-output/report.json`).

End of document. Source: `docs/SUPPORTING_DOCS.md` *. Polished submission version:* `docs/SUPPORTING_DOCS.pdf` *.*